Unit 4 : Basic Linux System Administration

4.1 System Administration: Role of Administrator, root- Administrator's Login, su: Acquiring superuser Status, Administrator's Privileges- passwd Commands, Task Scheduling using cron, Maintaining Security.
4.2 Operations: Startup and shutdown, System run levels
4.3 User management: User configuration and password file, Managing Users and Groups.

4.1 A. System Administration: Role of Administrator

What is Linux Administration?
Linux administration is about setting up disaster recovery, managing new system builds, creating a backup to restore data, Linux hardware management, managing storage, handling file systems, and managing the security of Linux systems. A big part of Linux administration is ensuring that Linux powered systems are stable and secure.

What Should a Linux Administrator Should Know?
Typically Linux system administrators are expected to handle Linux file systems, manage the root user, have a working knowledge of bash commands, and an ability to manage users.

What Are The Duties Of System Administrators In Linux?

Commonly, the Linux **administration role** typically involves:

1. Maintenance of a Linux environment.
2. Troubleshooting and providing support when there's an issue with Linux servers.
3. Analysis of log files(mainly error logs).
4. Support of LAN and web applications.
5. Creation of operational and project-specific solutions for the organization
6. Ability to proactively figure out ways to enforce strong security practices, and increase scalability of your Linux environment

4.1 B. System Administration: root-Administrator's Login, su: Acquiring superuser Status

The authorized user who has access to all files and commands within the system is known as the Superuser. The credentials of superuser are secured with the root password.
In UNIX, there are three main accounts: Root account, System account, User account.
The root account is referred to as the Superuser and that user has complete access to all the files and commands on a system.
This user can also be considered as a **system administrator** who has the ability to run any command without limitation.
The **root** user can do many things an ordinary user cannot, such as installing new software, changing the ownership of files, and managing other user accounts.
It is not recommended to use **root** for ordinary tasks, such as browsing the web, writing texts, e.g. A simple mistake can cause problems with the entire system, for example if you mistype a command.
It is advisable to create a normal user account for such tasks. If root permissions are needed, the *su* and *sudo* commands can be used.

For example,
If we try to bring the **eth0** interface down with an ordinary user, we will get the following message:

```
varsha:/$ ifconfig  eth0 down
ISOCSIFFLAGS: Operation not permitted
varsha:/$
```

To be able to perform the command above, we need to use the *su* or *sudo* command

```
varsha:/$ su ifconfig  eth0 down
password:
```

It will start bringing **eth0** interface down with an ordinary user having superuser root privileges.

The **root user**, also known as the **superuser** or **administrator**, is a special user account in Linux used for system administration. It is the most privileged user on the Linux system and it has access to all commands and files.

The **su** command lets you switch the current user to any other user. If you need to run a command as a different (non-root) user, use the **–l [username]** option to specify the user account.
Additionally, **su** can also be used to change to a different shell interpreter on the fly.

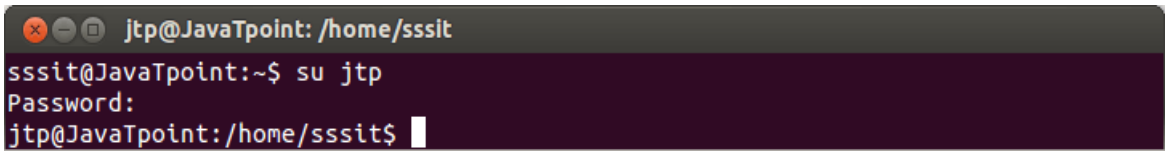
su is an older but more fully-featured command. It can duplicate the functionality of **sudo** by use of the **–c** option to pass a single command to the shell.
The su command allows you to run a shell as another user.

Syntax:

$su **<username>**

Example:

$su jtp



Look at the above snapshot, user account is changed from **sssit to jtp**.

1) su to root

You can change the user to root when you know the root password.

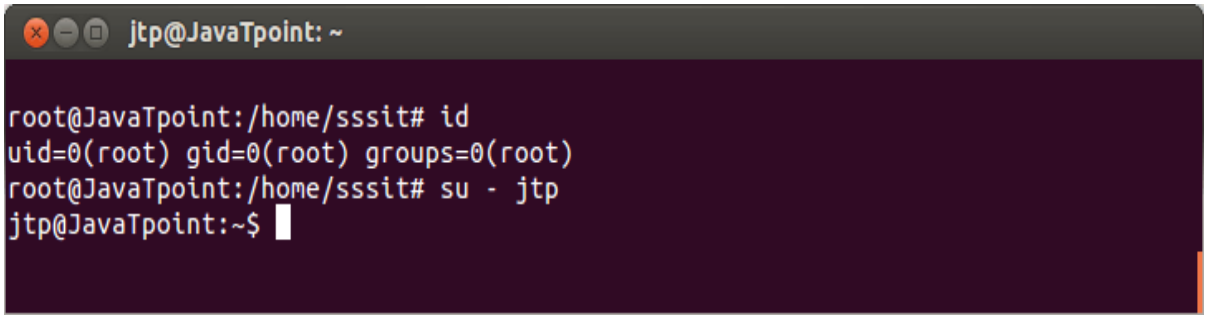
Syntax:

$su root

2) su as root

The root user can become any existing user without knowing that user's password. Otherwise, password is needed.
Example:

$su - sssit



Look at the above snapshot, it is asking for password while switching from user jtp to sssit.

Difference Between su and su – Command in Linux

As a new Linux user, you may always face confusion regarding the difference between su command and su – command. But before knowing about the difference between su and su – command, we need to make ourselves familiar with Linux User Environment

Linux User Environment:

Linux's systems are multi-user environments. Whenever Linux operating system creates a new shell session(after a new terminal being started on Linux) it started preparing an environment for itself.

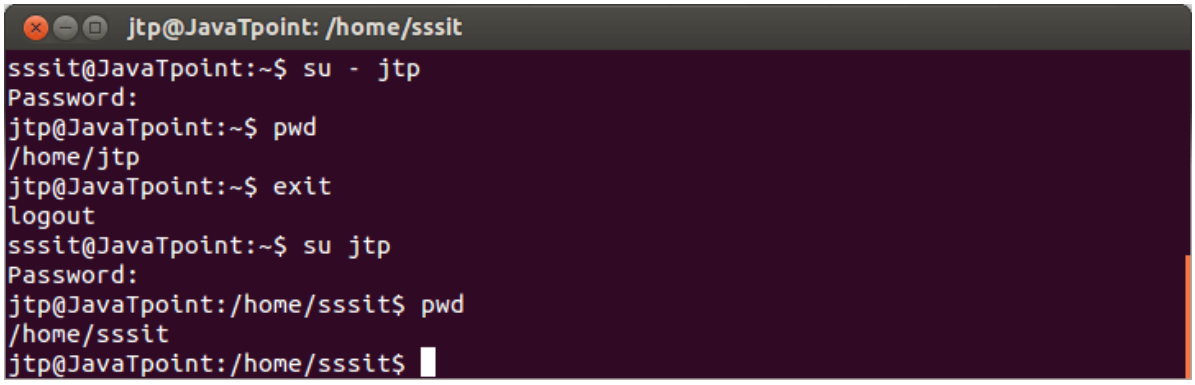
This environment basically holds the Environment variable (Environmental depends on shell type, Bash is generally used by most of the Linux distribution).

By default, the su command maintains the same shell environment. To access the target user's shell environment use the su command with (-) followed by the target user name.

Example:

$su - jtp

$su jtp



Look at the above snapshot, with the command **"su - jtp"** current shell environment is **/home/jtp** and user is also jtp. With the command **"su jtp"** current shell environment is **/home/sssit** and user is **sssit**.

4.1 C. Administrator's Privileges- passwd Commands

It enables users to change their password on a Linux system. Regular users on Linux can only change their password. But the root user (Super User) can change any user account password in Linux and there are no restrictions.

Type following passwd command to change your **own password**:

$ passwd

Sample Outputs:

(current) Linux password:
New Linux password:
Retype new Linux password:
passwd: password updated successfully

User will get Administrator's Privileges using su command.
Type following passwd command to change **other user password** even though you are in different user:

$ su passwd vivek

Sample Outputs:

```
password:                                   # type password for
superuser
(current) UNIX password:
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
```

4.1 D. System Administration: Task Scheduling using cron.

cron is a scheduling daemon that executes tasks at specified intervals. These tasks are called cron jobs and are mostly used to automate system maintenance or administration.

The cron is a software utility, offered by a Linux-like operating system that automates the scheduled task at a predetermined time.

It is a daemon process, which runs as a background process and performs the specified operations at the predefined time when a certain event or condition is triggered without the intervention of a user.

Dealing with a repeated task frequently is an intimidating task for the system administrator and thus he can schedule such processes to run automatically in the background at regular intervals of time by creating a list of those commands using cron.

It enables the users to execute the scheduled task on a regular basis unobtrusively like doing the backup every day at midnight, scheduling updates on a weekly basis, synchronizing the files at some regular interval.

The cron jobs can be scheduled to run by a minute, hour, day of the month, month, day of the week, or any combination of these.

For example, you could set a cron job to automate repetitive tasks such as backing up databases or data, updating the system with the latest security patches, checking the disk space usage , sending emails, and so on.

Cron checks for the scheduled job recurrently and when the scheduled time fields match the current time fields, the scheduled commands are executed.

It is started automatically from /etc/init.d on entering multi-user run levels.

Syntax:

cron [-f] [-l] [-L loglevel]

Options:
-f : Used to stay in foreground mode, and don't daemonize.
-l : This will enable the LSB compliant names for /etc/cron.d files.
-n : Used to add the FQDN in the subject when sending mails.
-L loglevel : This option will tell the cron what to log about the jobs with the following values:

1 : It will log the start of all cron jobs.
2 : It will log the end of all cron jobs.
4 : It will log all the failed jobs. Here the exit status will not equal to zero.
8 : It will log the process number of all the cron jobs.

crontab (cron table) is a text file that specifies the schedule of cron jobs. There are two types of crontab files.

- system-wide crontab files and
- individual user crontab files.

Users' crontab files are named according to the user's name, and their location varies by operating systems.

On Debian and Ubuntu files are stored in the /var/spool/cron/crontabs directory.
Although you can edit the user crontab files manually, it is recommended to use the crontab command.

The /etc/crontab file and the scripts inside the /etc/cron.d directory are system-wide crontab files that can be edited only by the system administrators.

Syntax:

crontab [-u user] file
crontab [-u user] { -l | -r | -e }
crontab [-u user] [-i] { -e | -l | -r }
crontab [-u user] [-l | -r | -e] [-i] [-s]

crontab is the program used to install, deinstall or list the tables used to drive the cron daemon in Vixie Cron.
A crontab file contains instructions to the cron daemon of the general form: "run this command at this time on this date". Each user can have their own crontab, and though these are files in **/var** directory, they are not intended to be edited directly.
If a -u option is given, it specifies the name of the user whose crontab is to be tweaked.If this option is not given, crontab examines "your" crontab, i.e., the crontab of the person executing the command.
Note that su can confuse crontab and that if you are running inside of su you should always use the -u option for safety's sake. cron file is used to install a new crontab from some named file or standard input if the pseudo-filename '-' is given.

Cron Table Format

```
*   *   *   *   *  Command_to_execute
|   |   |   |   |
|   |   |   |    Day of the Week ( 0 - 6 ) ( Sunday = 0 )
|   |   |   |
|   |   |   Month ( 1 - 12 )
|   |   |
|   |   Day of Month ( 1 - 31 )
|   |
|   Hour ( 0 - 23 )
|
Min ( 0 - 59 )
```

Specifying multiple values in a field

- The asterisk (*) operator specifies all possible values for a field. e.g. every hour or every day.
- The comma (,) operator specifies a list of values, for example: "1,3,4,7,8".
- The dash (-) operator specifies a range of values, for example: "1-6", which is equivalent to "1,2,3,4,5,6".
- The slash (/) operator, can be used to skip a given number of values. For example, "*/3" in the hour time field is equivalent to "0,3,6,9,12,15,18,21"; "*" specifies 'every hour' but the "/3" means that only the first, fourth, seventh...and such values given by "*" are used.

Cron will email to the user all output of the commands it runs, to silence this, redirect the output to a log file or to /dev/null.

Crontab Options

To Install or update job in crontab, use -e option:
$ crontab -e

To List Crontab entries, use -l option:
$ crontab -l

To Deinstall job from crontab, use -r option:
$ crontab -r

To Confirm Deinstall of job from crontab, use -i option:
$ crontab -i -r

To add SELINUX security to crontab file, use -s option:
$ crontab -s

To edit other user crontab, user -u option and specify username:
$ crontab -u username -e

To List other user crontab entries:
$ crontab -u username –l

EXAMPLES

To run /usr/bin/sample.sh at 12.59 every day and supress the output
59 12 * * * simon /usr/bin/sample.sh > /dev/null 2>&1

To run sample.sh everyday at 9pm (21:00)
0 21 * * * sample.sh 1>/dev/null 2>&1

To run sample.sh every Tuesday to Saturday at 1am (01:00)
0 1 * * 2-7 sample.sh 1>/dev/null 2>&1

To run sample.sh at 07:30, 09:30 13:30 and 15:30
30 07,09,13,15 * * * sample.sh

To run sample.sh at 07:30, 09:30 13:30 and 15:30
30 07,09,13,15 * * * sample.sh

To run sample.sh at 2am daily.

0 2 * * * sample.sh (This is widely used in cases of backup of files/databases etc on daily basis.)

To run sample.sh twice a day. say 5am and 5pm
0 5,17 * * * sample.sh

To run sample.sh every minutes.
* * * * * sample.sh

To run sample.sh every Sunday at 5 PM.
0 17 * * sun sample.sh

To run sample.sh every 10 minutes.
*/10 * * * * sample.sh

To run sample.sh on selected months.
* * * jan,may,aug * sample.sh

To run sample.sh selected days.
0 17 * * sun,fri sample.sh

To run sample.sh first sunday of every month.
0 2 * * sun [$(date +%d) -le 07] && sample.sh

To run sample.sh every four hours.
0 */4 * * * sample.sh

To run sample.sh every Sunday and Monday.
0 4,17 * * sun,mon sample.sh

To run sample.sh every 30 Seconds.
* * * * * sample.sh
* * * * * sleep 30; sample.sh

To run multiple jobs using single cron.
* * * * * sample1.sh; sample2.sh

To run sample.sh on yearly (@yearly).
@yearly sample.sh

To run sample.sh on monthly (@monthly).
@monthly sample.sh

To run sample.sh on Weekly (@weekly).
@weekly sample.sh

To run sample.sh on daily (@daily).
@daily sample.sh

To run sample.sh on hourly (@hourly).
@hourly sample.sh

To run sample.sh on system reboot (@reboot).
@reboot sample.sh

4.1 E. System Administration: Maintaining Security.

Linux is the most commonly used operating system for web-facing computers, running on nearly 75% of servers. Following are a few basic Linux hardening and Linux server security best practices

1. Use Strong and Unique Passwords. ...
2. Generate an SSH Key Pair. ...
3. Update Your Software Regularly. ...
4. Enable Automatic Updates. ...
5. Avoid Unnecessary Software. ...
6. Disable Booting from External Devices. ...
7. Close Hidden Open Ports. ...
8. Scan Log Files with Fail2ban.
9. Utilize Backups and Test Them Often
10. Perform Security Audits

4.2 A. Operations: Startup and shutdown

An operating system (OS) is the low-level software that manages resources, controls peripherals, and provides basic services to other software.

The following are the 6 high level stages of a typical Linux boot process (Startup process).

1. BIOS

- BIOS stands for Basic Input/Output System
- Performs some system integrity checks
- Searches, loads, and executes the boot loader program.
- It looks for boot loader in floppy, cd-rom, or hard drive. You can press a key (typically F12 of F2, but it depends on your system) during the BIOS startup to change the boot sequence.
- Once the boot loader program is detected and loaded into the memory, BIOS gives the control to it.
- So, in simple terms BIOS loads and executes the MBR boot loader.

2. MBR

- MBR stands for Master Boot Record.
- It is located in the 1st sector of the bootable disk. Typically /dev/hda, or /dev/sda

- MBR is less than 512 bytes in size. This has three components
    1) primary boot loader info in 1st 446 bytes
    2) partition table info in next 64 bytes
    3) mbr validation check in last 2 bytes.
- It contains information about GRUB (or LILO in old systems).
- So, in simple terms MBR loads and executes the GRUB boot loader.

3. GRUB

- GRUB stands for Grand Unified Bootloader.
- If you have multiple kernel images installed on your system, you can choose which one to be executed.
- GRUB displays a splash screen, waits for few seconds, if you don't enter anything, it loads the default kernel image as specified in the grub configuration file.
- GRUB has the knowledge of the filesystem (the older Linux loader LILO didn't understand filesystem).
- Grub configuration file is /boot/grub/grub.conf (/etc/grub.conf is a link to this). The following is sample grub.conf of CentOS.

```
#boot=/dev/sda
default=0
timeout=5
splashimage=(hd0,0)/boot/grub/splash.xpm.gz
hiddenmenu
title CentOS (2.6.18-194.el5PAE)
        root (hd0,0)
        kernel /boot/vmlinuz-2.6.18-194.el5PAE ro root=LABEL=/
        initrd /boot/initrd-2.6.18-194.el5PAE.img
```

- As you notice from the above info, it contains kernel and initrd image.
- So, in simple terms GRUB just loads and executes Kernel and initrd images.

4. Kernel

- Mounts the root file system as specified in the "root=" in grub.conf
- Kernel executes the /sbin/init program
- Since init was the 1st program to be executed by Linux Kernel, it has the process id (PID) of 1. Do a 'ps -ef | grep init' and check the pid.
- initrd stands for Initial RAM Disk.
- initrd is used by kernel as temporary root file system until kernel is booted and the real root file system is mounted. It also contains necessary drivers compiled inside, which helps it to access the hard drive partitions, and other hardware.

5. Init

1. Looks at the /etc/inittab file to decide the Linux run level.
2. Following are the available run levels
   - 0 – halt
   - 1 – Single user mode
   - 2 – Multiuser, without NFS
   - 3 – Full multiuser mode
   - 4 – unused
   - 5 – X11
   - 6 – reboot
3. Init identifies the default initlevel from /etc/inittab and uses that to load all appropriate program.
4. Execute 'grep initdefault /etc/inittab' on your system to identify the default run level
5. If you want to get into trouble, you can set the default run level to 0 or 6. Since you know what 0 and 6 means, probably you might not do that.
6. Typically you would set the default run level to either 3 or 5.

6. Runlevel programs

- When the Linux system is booting up, you might see various services getting started. For example, it might say "starting sendmail …. OK". Those are the runlevel programs, executed from the run level directory as defined by your run level.
- Depending on your default init level setting, the system will execute the programs from one of the following directories.
  - Run level 0 – /etc/rc.d/rc0.d/
  - Run level 1 – /etc/rc.d/rc1.d/
  - Run level 2 – /etc/rc.d/rc2.d/
  - Run level 3 – /etc/rc.d/rc3.d/
  - Run level 4 – /etc/rc.d/rc4.d/
  - Run level 5 – /etc/rc.d/rc5.d/
  - Run level 6 – /etc/rc.d/rc6.d/
- Please note that there are also symbolic links available for these directory under /etc directly. So, /etc/rc0.d is linked to /etc/rc.d/rc0.d.
- Under the /etc/rc.d/rc*.d/ directories, you would see programs that start with S and K.
- Programs starts with S are used during startup. S for startup.
- Programs starts with K are used during shutdown. K for kill.
- There are numbers right next to S and K in the program names. Those are the sequence number in which the programs should be started or killed.

- For example, S12syslog is to start the syslog deamon, which has the sequence number of 12. S80sendmail is to start the sendmail daemon, which has the sequence number of 80. So, syslog program will be started before sendmail.

There you have it. That is what happens during the Linux boot process.

4.2 B. Operations: shutdown

The **shutdown** command in Linux is used to shutdown the system in a safe way. You can shutdown the machine immediately, or schedule a shutdown using 24 hour format.
It brings the system down in a secure way.
When the shutdown is initiated, all logged-in users and processes are notified that the system is going down, and no further logins are allowed.
Only root user can execute shutdown command.

Syntax of shutdown Command

shutdown [OPTIONS] [TIME] [MESSAGE]

- options – Shutdown options such as halt, power-off (the default option) or reboot the system.
- time – The time argument specifies when to perform the shutdown process.
- message – The message argument specifies a message which will be broadcast to all users.

Options

-r : Requests that the system be rebooted after it has been brought down.
-h : Requests that the system be either halted or powered off after it has been brought down, with the choice as to which left up to the system.
-H : Requests that the system be halted after it has been brought down.
-P : Requests that the system be powered off after it has been brought down.
-c : Cancels a running shutdown. TIME is not specified with this option, the first argument is
-k : Only send out the warning messages and disable logins, do not actually bring the system down.

For example:
Add a **time** parameter, if you want to perform shutdown or reboot in N seconds. Here you can add broadcast a custom **message** to logged-in users. In this example, we are rebooting the machine in another 5 minutes.

 # shutdown -r +5 "To activate the latest Kernel"

Shutdown scheduled for Mon 2018-10-08 07:13:16 EDT, use 'shutdown -c' to cancel.
[root@vps138235 ~]#
Broadcast message from root@vps.2daygeek.com (Mon 2018-10-08 07:08:16 EDT):
To activate the latest Kernel
The system is going down for reboot at Mon 2018-10-08 07:13:16 EDT!

How to use shutdown

In it's simplest form when used without any argument, shutdown will power off the machine.

$sudo shutdown

How to shutdown the system at a specified time

The time argument can have two different formats. It can be an absolute time in the format hh:mm and relative time in the format +m where m is the number of minutes from now.

The following example will schedule a system shutdown at 05 A.M:

$sudo shutdown 05:00

The following example will schedule a system shutdown in 20 minutes from now:

$sudo shutdown +20

How to shutdown the system immediately

To shutdown your system immediately you can use +0 or its alias now:

$sudo shutdown now

How to broadcast a custom message

The following command will shut down the system in 10 minutes from now and notify the users with message "System upgrade":

$sudo shutdown +10 "System upgrade"

It is important to mention that when specifying a custom wall message you must specify a time argument too.

How to halt your system

This can be achieved using the -H option.

$shutdown -H

Halting means stopping all CPUs and powering off also makes sure the main power is disconnected.

How to make shutdown power-off machine

Although this is by default, you can still use the -P option to explicitly specify that you want shutdown to power off the system.

$shutdown -P

How to reboot using shutdown

For reboot, the option is -r.

$shutdown -r

You can also specify a time argument and a custom message:

$shutdown -r +5 "Updating Your System"

The command above will reboot the system after 5 minutes and broadcast Updating Your System”

How to cancel a scheduled shutdown

If you have scheduled a shutdown and you want to cancel it you can use the -c argument:

sudo shutdown -c

When canceling a scheduled shutdown, you cannot specify a time argument, but you can still broadcast a message that will be sent to all users.

$sudo shutdown -c "Canceling the reboot"

4.2 B. Operations: System run levels

Run Levels in Linux

A run level is a state of init and the whole system that defines what system services are operating. Run levels are identified by numbers. Some system administrators use run levels to define which subsystems are working, e.g., whether X is running, whether the network is operational, and so on.

- Whenever a LINUX system boots, firstly the **init** process is started which is actually responsible for running other start scripts which mainly involves initialization of you hardware, bringing up the network, starting the graphical interface.
- Now, the **init** first finds the default **runlevel** of the system so that it could run the start scripts corresponding to the default run level.
- A **runlevel** can simply be thought of as the state your system enters like if a system is in a single-user mode it will have a **runlevel 1** while if the system is in a multi-user mode it will have a **runlevel 5**.

- A **runlevel** in other words can be defined as a **preset single digit integer** for defining the operating state of your LINUX or UNIX-based operating system. Each runlevel designates a different system configuration and allows access to different combination of **processes**.

The important thing to note here is that there are differences in the runlevels according to the operating system. The standard **LINUX kernel** supports these seven different runlevels :

Run Level	Mode	Action
0	Halt	Shuts down system
1	Single-User Mode	Does not configure network interfaces, start daemons, or allow non-root logins
2	Multi-User Mode	Does not configure network interfaces or start daemons.
3	Multi-User Mode with Networking	Starts the system normally.
4	Undefined	Not used/User-definable
5	X11	As runlevel 3 + display manager(X)
6	Reboot	Reboots the system

- 0 – System halt *i.e* the system can be safely powered off with no activity.
- 1 – Single user mode.
- 2 – Multiple user mode with no NFS(network file system).
- 3 – Multiple user mode under the command line interface and not under the graphical user interface.
- 4 – User-definable.
- 5 – Multiple user mode under GUI (graphical user interface) and this is the standard runlevel for most of the LINUX based systems.
- 6 – Reboot which is used to restart the system.

By default most of the LINUX based system boots to runlevel 3 or runlevel 5. In addition to the standard runlevels, users can modify the preset runlevels or even create new ones according to the requirement. Runlevels 2 and 4 are used for user defined runlevels and runlevel 0 and 6 are used for halting and rebooting the system.

Obviously the start scripts for each run level will be different performing different tasks. These start scripts corresponding to each run level can be found in special files present under **rc sub directories**.

At **/etc/rc.d** directory there will be either a set of files named **rc.0, rc.1, rc.2, rc.3, rc.4, rc.5** and **rc.6,** or a set of directories named **rc0.d, rc1.d, rc2.d, rc3.d, rc4.d, rc5.d** and **rc6.d**.

For example, run level 1 will have its start script either in file /etc/rc.d/rc.1 or any files in the directory /etc/rc.d/rc1.d.

Changing runlevel

init is the program responsible for altering the run level which can be called using **telinit** command. "The telinit command (really just a symbolic link to init) enables you to specify a desired run level, causing the termination of all system processes that shouldn't exist in that run level, and starting all processes that should be running."
For example, to change a runlevel from 3 to runlevel 5 which will actually allow the GUI to be started in multi-user mode the **telinit** command can be used as :

/*using telinit to change
runlevel from 3 to 5*/
telinit 5

NOTE : The changing of runlevels is a task for the super user and not the normal user that's why it is necessary to be logged in as super user for the successful execution of the above telinit command or you can use **sudo** command as :

// using sudo to execute telinit
sudo telinit 5

The default runlevel for a system is specified in **/etc/initab** file which will have an entry **id : 5 : initdefault** if the default runlevel is set to 5 or will have an entry **id : 3 : initdefault** if the default runlevel is set to 3.

Need for changing the runlevel

- There can be a situation when you may find trouble in **logging in** in case you **don't remember the password or because of the corrupted /etc/passwd** file (have all the user names and passwords), in this case the problem can be solved by booting into a **single user mode *i.e* runlevel 1.**
- You can easily halt the system by changing the runlevel to 0 by using **telinit 0**.

To View Runlevel Config File
cat /etc/inittab

To Change Runlevel Configuration File
vi /etc/inittab

To check Current Run Level

who –r or

runlevel

To change Run Level

init 1

On most Linux server system default run level is 3 and on most Linux Desktop system default run level is 5.

4.3 A. User management: password file

The **passwd** command changes passwords for user accounts. A normal user may only change the password for his/her own account, while the superuser may change the password for any account. passwd also changes the account or associated password validity period.

Syntax:

passwd [options] [LOGIN]

OPTIONS

TAG	DESCRIPTION
-a, --all	This option can be used only with -S and causes show status for all users.
-d, --delete	Delete a user's password (make it empty). This is a quick way to disable a password for an account. It will set the named account passwordless.
-e, --expire	Immediately expire an account's password. This in effect can force a user to change his/her password at the user's next login.
-h, --help	Display help message and exit.
-g, --noheadings	Do not print a header line.
-h, --help	Display help text and exit.
-i, --inactive INACTIVE	This option is used to disable an account after the password has been expired for a number of days. After a user account has had an expired password for INACTIVE days, the user may no longer sign on to the account.
-k, --keep-	Indicate password change should be performed only for expired

tokens	authentication tokens (passwords). The user wishes to keep their non-expired tokens as before.
-l, --lock	Lock the password of the named account. This option disables a password by changing it to a value which matches no possible encrypted value (it adds a '!' at the beginning of the password). Note that this does not disable the account. The user may still be able to login using another authentication token (e.g. an SSH key). To disable the account, administrators should use usermod --expiredate 1 (this set the account's expire date to Jan 2, 1970). Users with a locked password are not allowed to change their password.
-n, --mindays MIN_DAYS	Set the minimum number of days between password changes to MIN_DAYS. A value of zero for this field indicates that the user may change his/her password at any time.
-q, --quiet	Quiet mode.
-r, --repository REPOSITORY	change password in REPOSITORY repository
-R, --root CHROOT_DIR	Apply changes in the CHROOT_DIR directory and use the configuration files from the CHROOT_DIR directory.
-S, --status	Display account status information. The status information consists of 7 fields. The first field is the user's login name. The second field indicates if the user account has a locked password (L), has no password (NP), or has a usable password (P). The third field gives the date of the last password change. The next four fields are the minimum age, maximum age, warning period, and inactivity period for the password. These ages are expressed in days.
-u, --unlock	Unlock the password of the named account. This option re-enables a password by changing the password back to its previous value (to the value before using the -l option).
-w, --warndays WARN_DAYS	Set the number of days of warning before a password change is required. The WARN_DAYS option is the number of days prior to the password expiring that a user will be warned that his/her password is about to expire.

-x, --maxdays MAX_DAYS	Set the maximum number of days a password remains valid. After MAX_DAYS, the password is required to be changed.

EXAMPLES

Example-1:
Change your own password:
$ passwd

output:
$ passwd
Changing password for ubuntu.
(current) UNIX password:
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully

Administrator's Privileges- passwd Commands

Example-2:
Change the password for the user named username:
$ sudo passwd username
output:
$ sudo passwd user1
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully

Example-3:
Check the status of the password for the user named user1:
$ sudo passwd -S user1
output:
user1 P 05/13/2014 2 365 7 28
Here, we see the user's name (user1), followed by a P, indicating that his password is currently valid and usable.
The password will expire on May 5, 2014. user1 cannot change his password more often than every 2 days, and must change the
password every 365 days. He will be warned 7 days before a required password change, and if he allows his password to expire,

his account will be disabled 28 days later.

Example-4:

Checks the password status for all user accounts, system-wide:

$ sudo passwd -S -a

output:

root L 12/27/2016 0 99999 7 -1
daemon L 08/05/2015 0 99999 7 -1
bin L 08/05/2015 0 99999 7 -1
sys L 08/05/2015 0 99999 7 -1
sync L 08/05/2015 0 99999 7 -1
games L 08/05/2015 0 99999 7 -1
man L 08/05/2015 0 99999 7 -1
lp L 08/05/2015 0 99999 7 -1
mail L 08/05/2015 0 99999 7 -1
news L 08/05/2015 0 99999 7 -1
uucp L 08/05/2015 0 99999 7 -1
proxy L 08/05/2015 0 99999 7 -1
www-data L 08/05/2015 0 99999 7 -1
backup L 08/05/2015 0 99999 7 -1
list L 08/05/2015 0 99999 7 -1
irc L 08/05/2015 0 99999 7 -1
gnats L 08/05/2015 0 99999 7 -1
nobody L 08/05/2015 0 99999 7 -1
libuuid L 08/05/2015 0 99999 7 -1
syslog L 08/05/2015 0 99999 7 -1
messagebus L 12/27/2016 0 99999 7 -1
dnsmasq L 12/27/2016 0 99999 7 -1
landscape L 12/27/2016 0 99999 7 -1
sshd L 12/27/2016 0 99999 7 -1
libvirt-qemu L 12/27/2016 0 99999 7 -1
libvirt-dnsmasq L 12/27/2016 0 99999 7 -1
ubuntu P 01/06/2017 0 99999 7 -1
user1 P 01/06/2017 0 99999 7 -1

Example-5:

Lock the password for user user1. user1 will be unable to log in until a system administrator unlocks it:

$ sudo passwd -l user1

output:

passwd: password expiry information changed.

Example-6:
Unlock user1's password. It will automatically be reset to whatever it was before it was locked, and user1 will be able to log in again:
$ sudo passwd -u user1
output:
passwd: password expiry information changed.

Example-7:
Expire user1's password. The next time he logs in, he will be required to set a new password.
$ sudo passwd -e user1
output:
passwd: password expiry information changed.
next time user login:
$ su user1
Password:
You are required to change your password immediately (root enforced)
Changing password for user1.
(current) UNIX password:
Enter new UNIX password:
Retype new UNIX password:

Example-8:
Delete a user's password (make it empty). This is a quick way to disable a password for an account. It will set the named account passwordless.
$ sudo passwd -d user1
output:
passwd: password expiry information changed.

Using the /etc/passwd file

Traditionally, the /etc/passwd file is used to keep track of every registered user that has access to a system.

The /etc/passwd file is a colon-separated file that contains the following information:

1. User name
2. Encrypted password
3. User ID number (UID)
4. User's group ID number (GID)
5. Full name of the user (GECOS)
6. User home directory
7. Login shell

The following is an example of an /etc/passwd file:

root:!:0:0::/:/usr/bin/ksh
daemon:!:1:1::/etc:
bin:!:2:2::/bin:
sys:!:3:3::/usr/sys:
adm:!:4:4::/var/adm:
uucp:!:5:5::/usr/lib/uucp:
guest:!:100:100::/home/guest:
nobody:!:4294967294:4294967294::/:
lpd:!:9:4294967294::/:
lp:*:11:11::/var/spool/lp:/bin/false
invscout:*:200:1::/var/adm/invscout:/usr/bin/ksh
nuucp:*:6:5:uucp login user:/var/spool/uucppublic:/usr/sbin/uucp/uucico
paul:!:201:1::/home/paul:/usr/bin/ksh
jdoe:*:202:1:John Doe:/home/jdoe:/usr/bin/ksh

The /etc/security/passwd file by default, which is only readable by the root user. The password field in the /etc/passwd file is used to signify whether a password exists or whether the account is blocked.

The /etc/passwd file is owned by the root user and must be readable by all the users, but only the root user has writable permissions, which are shown as -rw-r--r--.

- If a user ID has a password, then the password field will have an **!** (exclamation point).
- If the user ID does not have a password, then the password field will have an ***** (asterisk).

The encrypted passwords are stored in the /etc/security/passwd file.

The following example contains the last four entries in the /etc/security/passwd file based on the entries from the /etc/passwd file shown previously.

```
guest:
        password = *

nobody:
        password = *

lpd:
        password = *

paul:
        password = eacVScDKri4s6
        lastupdate = 1026394230
        flags = ADMCHG
```

The user ID jdoe does not have an entry in the /etc/security/passwd file because it does not have a password set in the /etc/passwd file.

The consistency of the /etc/passwd file can be checked using the pwdck command. The pwdck command verifies the correctness of the password information in the user database files by checking the definitions for all of the users or for specified users.

4.3 B. User management: User configuration

There are three types of accounts on a Unix system –

1. **Root account**

This is also called **superuser** and would have complete and unfettered control of the system. A superuser can run any commands without any restriction. This user should be assumed as a system administrator.

2. **System accounts**

System accounts are those needed for the operation of system-specific components for example mail accounts and the **sshd** accounts. These accounts are usually needed for some specific function on your system, and any modifications to them could adversely affect the system.

3. **User accounts**

User accounts provide interactive access to the system for users and groups of users. General users are typically assigned to these accounts and usually have limited access to critical system files and directories.

Unix supports a concept of *Group Account* which logically groups a number of accounts. Every account would be a part of another group account. A Unix group plays important role in handling file permissions and process management.

Managing Users and Groups

There are four main user administration files –

- **/etc/passwd** – Keeps the user account and password information. This file holds the majority of information about accounts on the Unix system.
- **/etc/shadow** – Holds the encrypted password of the corresponding account. Not all the systems support this file.
- **/etc/group** – This file contains the group information for each account.
- **/etc/gshadow** – This file contains secure group account information.

Check all the above files using the **cat** command.

The following table lists out commands that are available on majority of Unix systems to create and manage accounts and groups –

Sr.No.	Command & Description
1	**useradd** Adds accounts to the system
2	**usermod** Modifies account attributes
3	**userdel** Deletes accounts from the system
4	**groupadd** Adds groups to the system
5	**groupmod** Modifies group attributes
6	**groupdel** Removes groups from the system

Create a Group

We will now understand how to create a group. For this, we need to create groups before creating any account otherwise, we can make use of the existing groups in our system. We have all the groups listed in ***/etc/groups*** file.
All the default groups are system account specific groups and it is not recommended to use them for ordinary accounts. So, following is the syntax to create a new group account –

$groupadd [-g gid [-o]] [-r] [-f] groupname

The following table lists out the parameters –

Sr.No.	Option & Description
1	**-g GID** The numerical value of the group's ID
2	**-o** This option permits to add group with non-unique GID
3	**-r** This flag instructs **groupadd** to add a system account
4	**-f** This option causes to just exit with success status, if the specified group already exists. With -g, if the specified GID already exists, other (unique) GID is chosen
5	**groupname** Actual group name to be created

If you do not specify any parameter, then the system makes use of the default values.
Following example creates a *developers* group with default values, which is very much acceptable for most of the administrators.

$ groupadd developers

Modify a Group

To modify a group, use the **groupmod** syntax –

```
$ groupmod -n new_modified_group_name old_group_name
```

To change the developers_2 group name to developer, type –

```
$ groupmod -n developer developer_2
```

Here is how you will change the -- GID to 545 –

```
$ groupmod -g 545 developer
```

Delete a Group

We will now understand how to delete a group. To delete an existing group, all you need is the **groupdel command** and the **group name**. To delete the financial group, the command is –

```
$ groupdel developer
```

This removes only the group, not the files associated with that group. The files are still accessible by their owners.

Create an Account

Let us see how to create a new account on your Unix system. Following is the syntax to create a user's account –

```
useradd -d homedir -g groupname -m -s shell -u userid accountname
```

The following table lists out the parameters –

Sr.No.	Option & Description
1	**-d homedir** Specifies home directory for the account
2	**-g groupname** Specifies a group account for this account
3	**-m** Creates the home directory if it doesn't exist
4	**-s shell** Specifies the default shell for this account

5	**-u userid** You can specify a user id for this account
6	**accountname** Actual account name to be created

If you do not specify any parameter, then the system makes use of the default values. The **useradd** command modifies the **/etc/passwd**, **/etc/shadow**, and **/etc/group** files and creates a home directory.

Following is the example that creates an account ***mcmohd***, setting its home directory to ***/home/mcmohd*** and the group as ***developers***. This user would have Korn Shell assigned to it.

```
$ useradd -d /home/mcmohd -g developers -s /bin/ksh mcmohd
```

Before issuing the above command, make sure you already have the *developers* group created using the ***groupadd*** command.
Once an account is created you can set its password using the **passwd** command as follows –

```
$ passwd mcmohd20
```

```
Changing password for user mcmohd20.
New Linux password:
Retype new Linux password:
passwd: all authentication tokens updated successfully.
```

When you type ***passwd accountname***, it gives you an option to change the password, provided you are a superuser. Otherwise, you can change just your password using the same command but without specifying your account name.

Modify an Account

The **usermod** command enables you to make changes to an existing account from the command line. It uses the same arguments as the **useradd** command, plus the -l argument, which allows you to change the account name.

For example, to change the account name ***mcmohd*** to ***mcmohd20*** and to change home directory accordingly, you will need to issue the following command –

```
$ usermod -d /home/mcmohd20 -m -l mcmohd mcmohd20
```

Delete an Account

The **userdel** command can be used to delete an existing user. This is a very dangerous command if not used with caution.

There is only one argument or option available for the command **.r**, for removing the account's home directory and mail file.

For example, to remove account *mcmohd20*, issue the following command –

```
$ userdel -r mcmohd20
```

If you want to keep the home directory for backup purposes, omit the **-r** option. You can remove the home directory as needed at a later time.